Mobile Learning Application Development

Dart Programming Language
Lecture 4

Aditya Rajbongshi
Assistant Professor
Dept. of Educational Technology and Engineering
University of Frontier Technology, Bangladesh

Dart Function

Dart function is a set of codes that together perform a specific task. It is used to break the large code into smaller modules and reuse it when needed. Functions make the program more readable and easy to debug. It improves the modular approach and enhances the code reusability.

❖ Advantages of Functions

- It increases the module approach to solve the problems.
- It enhances the re-usability of the program.
- We can do the coupling of the programs.
- It optimizes the code.
- It makes debugging easier.
- It makes development easy and creates less complexity.

Defining a Function

A function can be defined by providing the name of the function with the appropriate parameter and return type. A function contains a set of statements which are called function body.

Syntax:

```
return_type func_name (parameter_list):
{
    //statement(s)
   return value;
}
```

The general syntax of the defining function

return_type - It can be any data type such as void, integer, float, etc. The return type must be matched with the returned value of the function.

func_name - It should be an appropriate and valid identifier.

parameter_list - It denotes the list of the parameters, which is necessary when we called a function.

return value - A function returns a value after complete its execution.

Calling a Function

After creating a function, we can call or invoke the defined function inside the main() function body. A function is invoked simply by its name with a parameter list, if any.

Syntax:

fun_name(<argument_list>);

or

variable = function_name(argument);

```
void main(){
   int sum(int a, int b){
            int result;
            result = a+b;
            return result;
}
var c = sum(30,20);
print("The sum of two numbers is: ${c}");
}
```

Function with No Parameter and Return Value

As we discussed earlier, the parameters are optional to pass while defining a function. We can create a function without parameter return value.

Syntax:

```
return_type func_name()
{
    //Statement(s);
    return value;
}
```

```
void main(){

String greetings(){
   return "Welcome to UFTB";
}
// Calling function inside print statement
print(greetings());
}
```

Function with No Parameter and Return Value

We are creating a function to find the given number is even or odd. Let's understand the following example.

```
void main()
{
  void number(int n){

          if (n%2 ==0){
                print("The given number is even");
           }
          else {
                print("The given number is odd");
           }
}
   number(20);
}
```

Dart Anonymous Function

Dart also provides the facility to specify a nameless function or function without a name. This type of function is known as an **anonymous function, lambda, or closure**. An anonymous function behaves the same as a regular function, but it does not have a name with it. It can have zero or any number of arguments with an optional type annotation.

Syntax:

```
(parameter_list) {
        statement(s)
}
```

```
void main() {
  var list = ["James","Patrick","Mathew"
,"Tom"];
  print("Example of anonymous function
");
  list.forEach((item) {
      print('${list.indexOf(item)}: $item');

});
}
```

The main() function

- The main() function is the top-level function of the Dart.
- It is the most important and vital function of the Dart programming language.
- The execution of the programming starts with the main() function.
- The main() function can be used only once in a program.

```
void main() {
  int mul(int a, int b){
      int c = a*b;
      return c;
}
print("The multiplication of two numbers:
 ${mul(10,20)}");
}
```

Function Recursion

- Dart Recursion is the method where a function calls itself as its subroutine.
- It is used to solve the complex problem by dividing it into sub-part.
- A function which is called itself again and again or recursively, then this process is called recursion.

```
int factorial(int num){
  if(num<=1) {
        return 1;
}
  else{
         return num*factorial(num-1);
  }
}
void main() {
  var num = 5;
  var fact = factorial(num);
  print("Factorial Of 5 is: ${fact}");
}
```